Oracle SQL & PL/SQL Database Documentation

Live Stock & Cryptocurrency Portfolio Management System — SQL*Plus / Oracle Enterprise Specification

This document contains every SQL Data Definition Language (DDL), Data Manipulation Language (DML), Analytical/Reporting query, and PL/SQL Stored Procedure, Function, and Trigger converted and optimized specifically for **Oracle Database (12c / 19c / 21c / 23c)** and **SQL*Plus**. Each statement adheres strictly to Oracle syntax specifications, data types, and procedural standards.

Conversion Rules & Architecture Conventions:

- **Identifiers & Primary Keys**: Converted PostgreSQL UUID / uuid_generate_v4() to Oracle RAW(16) DEFAULT SYS_GUID().
- **Character Types**: Converted VARCHAR(n) to VARCHAR2(n).
- **Boolean Logic**: Mapped BOOLEAN to NUMBER(1) with explicit CHECK (col IN (0, 1)) constraints.
- **JSON Data**: Converted JSONB to CLOB with an explicit CHECK (metadata IS JSON) constraint.
- **Timestamps**: Replaced DEFAULT CURRENT_TIMESTAMP with DEFAULT SYSTIMESTAMP.
- **Upsert Operations**: Replaced PostgreSQL INSERT ... ON CONFLICT with ANSI/Oracle MERGE INTO ... USING dual.
- **Returning Clauses**: Converted RETURNING ... INTO to PL/SQL blocks with bind variables and DBMS_OUTPUT.PUT_LINE.
- **Paging Syntax**: Converted LIMIT n OFFSET m to ANSI standard OFFSET m ROWS FETCH NEXT n ROWS ONLY.
- **Table Functions**: Converted PostgreSQL RETURNS TABLE to Oracle PL/SQL functions returning SYS_REFCURSOR.

Part 1: Table Creation (DDL) Queries

1. Global Identity Support Note

```
-- Oracle utilizes the built-in SYS_GUID() function returning RAW(16)
-- No third-party extension (e.g., uuid-ossp) is required.
SELECT SYS_GUID() AS generated_uuid FROM dual;
```

***Purpose**: Verifies built-in Oracle 128-bit globally unique identifier (GUID/UUID) generation.*
***ER Diagram Check**: ■ Matches*

2. Create Currency Rates Table

```
CREATE TABLE currency_rates (
    currency_code VARCHAR2(10) PRIMARY KEY,
    rate_to_usd NUMBER(18, 6) NOT NULL,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT chk_cr_rate CHECK (rate_to_usd > 0)
);
```

***Purpose**: Stores reference exchange rates for world currencies relative to USD.*
***ER Diagram Check**: ■ Matches*

3. Create Users Table

```
CREATE TABLE users (
    user_id RAW(16) DEFAULT SYS_GUID() PRIMARY KEY,
    email VARCHAR2(255) UNIQUE NOT NULL,
    password_hash VARCHAR2(255) NOT NULL,
    base_currency VARCHAR2(10) DEFAULT 'USD' NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT fk_users_currency FOREIGN KEY (base_currency) REFERENCES currency_rates (currency_code)
);
```

***Purpose**: Stores user identities, credentials, base currency preferences, and creation timestamps.*
***ER Diagram Check**: ■ Matches*

4. Create Exchanges Table

```
CREATE TABLE exchanges (
    exchange_id RAW(16) DEFAULT SYS_GUID() PRIMARY KEY,
    name VARCHAR2(100) NOT NULL,
    country VARCHAR2(100) NOT NULL,
    url VARCHAR2(500)
);
```

***Purpose**: Stores financial exchanges and trading venues where assets are listed.*
***ER Diagram Check**: ■ Matches*

5. Create Assets Table

```
CREATE TABLE assets (
    asset_id RAW(16) DEFAULT SYS_GUID() PRIMARY KEY,
    symbol VARCHAR2(20) UNIQUE NOT NULL,
    name VARCHAR2(150) NOT NULL,
    type VARCHAR2(20) NOT NULL,
    network VARCHAR2(100),
    contract_address VARCHAR2(255),
    ticker_exchange VARCHAR2(50),
    sector VARCHAR2(100),
    exchange_id RAW(16),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT chk_assets_type CHECK (type IN ('crypto', 'stock')),
    CONSTRAINT fk_assets_exchange FOREIGN KEY (exchange_id) REFERENCES exchanges (exchange_id) ON DELETE SET NULL
);
```

***Purpose**: Catalogs global tradeable assets with polymorphic fields for crypto networks and stock sectors.*
***ER Diagram Check**: ■ Matches*

6. Create Ticker Stats Table

```
CREATE TABLE ticker_stats (
    asset_id RAW(16) PRIMARY KEY,
    price_per_unit NUMBER(18, 6) DEFAULT 0.0 NOT NULL,
    volume_24h NUMBER(24, 2) DEFAULT 0.0,
    high_24h NUMBER(18, 6) DEFAULT 0.0,
    low_24h NUMBER(18, 6) DEFAULT 0.0,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT fk_ticker_stats_asset FOREIGN KEY (asset_id) REFERENCES assets (asset_id) ON DELETE CASCADE
);
```

***Purpose**: Stores 1:1 24-hour market metrics and live statistics for cataloged assets.*
***ER Diagram Check**: ■ Matches*

7. Create Portfolios Table

```
CREATE TABLE portfolios (
    portfolio_id RAW(16) DEFAULT SYS_GUID() PRIMARY KEY,
    user_id RAW(16) NOT NULL,
    name VARCHAR2(100) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT fk_portfolios_user FOREIGN KEY (user_id) REFERENCES users (user_id) ON DELETE CASCADE
);
```

***Purpose**: Defines user portfolios that aggregate holdings and investment positions.*
***ER Diagram Check**: ■ Matches*

8. Create Holdings Table

```
CREATE TABLE holdings (
    holding_id RAW(16) DEFAULT SYS_GUID() PRIMARY KEY,
    portfolio_id RAW(16) NOT NULL,
    asset_id RAW(16),
    symbol VARCHAR2(20) NOT NULL,
    name VARCHAR2(150) NOT NULL,
    type VARCHAR2(20) NOT NULL,
    quantity NUMBER(24, 8) NOT NULL,
    average_buy_price NUMBER(18, 6) NOT NULL,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT chk_holdings_type CHECK (type IN ('crypto', 'stock')),
    CONSTRAINT chk_holdings_quantity CHECK (quantity > 0),
    CONSTRAINT chk_holdings_avg_price CHECK (average_buy_price >= 0),
    CONSTRAINT fk_holdings_portfolio FOREIGN KEY (portfolio_id) REFERENCES portfolios (portfolio_id) ON DELETE CASCADE,
    CONSTRAINT fk_holdings_asset FOREIGN KEY (asset_id) REFERENCES assets (asset_id) ON DELETE SET NULL,
    CONSTRAINT uq_portfolio_symbol UNIQUE (portfolio_id, symbol)
);
```

***Purpose**: Stores individual asset positions held within a portfolio with purchase cost bases.*
***ER Diagram Check**: ■ Matches*

9. Create Transactions Table

```
CREATE TABLE transactions (
    transaction_id RAW(16) DEFAULT SYS_GUID() PRIMARY KEY,
    user_id RAW(16) NOT NULL,
    portfolio_id RAW(16) NOT NULL,
    asset_id RAW(16),
    symbol VARCHAR2(20) NOT NULL,
    type VARCHAR2(20) NOT NULL,
    quantity NUMBER(24, 8) NOT NULL,
    price_at_transaction NUMBER(18, 6) NOT NULL,
    network_hash VARCHAR2(255),
    timestamp TIMESTAMP WITH TIME ZONE DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT chk_tx_type CHECK (type IN ('BUY', 'SELL', 'TRANSFER_IN', 'TRANSFER_OUT')),
    CONSTRAINT chk_tx_quantity CHECK (quantity > 0),
    CONSTRAINT chk_tx_price CHECK (price_at_transaction >= 0),
    CONSTRAINT fk_tx_user FOREIGN KEY (user_id) REFERENCES users (user_id) ON DELETE CASCADE,
    CONSTRAINT fk_tx_portfolio FOREIGN KEY (portfolio_id) REFERENCES portfolios (portfolio_id) ON DELETE CASCADE,
    CONSTRAINT fk_tx_asset FOREIGN KEY (asset_id) REFERENCES assets (asset_id) ON DELETE SET NULL
);
```

***Purpose**: Maintains an immutable ledger of buy, sell, and transfer executions.*
***ER Diagram Check**: ■ Matches*

10. Create Portfolio Performance Table

```
CREATE TABLE portfolio_performance (
    performance_id RAW(16) DEFAULT SYS_GUID() PRIMARY KEY,
    portfolio_id RAW(16) NOT NULL,
    total_value NUMBER(18, 2) NOT NULL,
    total_profit_loss NUMBER(18, 2) NOT NULL,
    total_profit_loss_pct NUMBER(8, 4) NOT NULL,
    timestamp TIMESTAMP WITH TIME ZONE DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT fk_perf_portfolio FOREIGN KEY (portfolio_id) REFERENCES portfolios (portfolio_id) ON DELETE CASCADE
);
```

***Purpose**: Records periodic valuation and profit/loss snapshots for historical tracking.*
***ER Diagram Check**: ■ Matches*

11. Create Watchlists Table

```
CREATE TABLE watchlists (
    watchlist_id RAW(16) DEFAULT SYS_GUID() PRIMARY KEY,
    user_id RAW(16) NOT NULL,
    name VARCHAR2(100) NOT NULL,
    creation_date TIMESTAMP WITH TIME ZONE DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT fk_watchlists_user FOREIGN KEY (user_id) REFERENCES users (user_id) ON DELETE CASCADE
);
```

***Purpose**: Stores custom user watchlists for tracking market symbols.*
***ER Diagram Check**: ■ Matches*

12. Create Watchlist Items Table

```
CREATE TABLE watchlist_items (
    item_id RAW(16) DEFAULT SYS_GUID() PRIMARY KEY,
    watchlist_id RAW(16) NOT NULL,
    symbol VARCHAR2(20) NOT NULL,
    asset_id RAW(16),
    added_at TIMESTAMP WITH TIME ZONE DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT fk_wl_items_watchlist FOREIGN KEY (watchlist_id) REFERENCES watchlists (watchlist_id) ON DELETE CASCADE,
    CONSTRAINT fk_wl_items_asset FOREIGN KEY (asset_id) REFERENCES assets (asset_id) ON DELETE SET NULL,
    CONSTRAINT uq_watchlist_symbol UNIQUE (watchlist_id, symbol)
);
```

***Purpose**: Represents tracked asset entries associated with a watchlist.*
***ER Diagram Check**: ■ Matches*

13. Create Alerts Table

```
CREATE TABLE alerts (
    alert_id RAW(16) DEFAULT SYS_GUID() PRIMARY KEY,
    user_id RAW(16) NOT NULL,
    symbol VARCHAR2(20) NOT NULL,
    asset_id RAW(16),
    threshold NUMBER(18, 6) NOT NULL,
    direction VARCHAR2(10) NOT NULL,
    enabled NUMBER(1) DEFAULT 1 NOT NULL,
    triggered_at TIMESTAMP WITH TIME ZONE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT chk_alerts_threshold CHECK (threshold > 0),
    CONSTRAINT chk_alerts_direction CHECK (direction IN ('ABOVE', 'BELOW')),
    CONSTRAINT chk_alerts_enabled CHECK (enabled IN (0, 1)),
    CONSTRAINT fk_alerts_user FOREIGN KEY (user_id) REFERENCES users (user_id) ON DELETE CASCADE,
    CONSTRAINT fk_alerts_asset FOREIGN KEY (asset_id) REFERENCES assets (asset_id) ON DELETE SET NULL
);
```

***Purpose**: Defines price threshold trigger rules created by users.*
***ER Diagram Check**: ■ Matches*

14. Create Notifications Table

```
CREATE TABLE notifications (
    notification_id RAW(16) DEFAULT SYS_GUID() PRIMARY KEY,
    alert_id RAW(16) NOT NULL,
    user_id RAW(16) NOT NULL,
    channel VARCHAR2(20) NOT NULL,
    status VARCHAR2(20) NOT NULL,
    sent_at TIMESTAMP WITH TIME ZONE DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT chk_notif_channel CHECK (channel IN ('EMAIL', 'SMS', 'PUSH', 'IN_APP')),
    CONSTRAINT chk_notif_status CHECK (status IN ('PENDING', 'SENT', 'FAILED')),
    CONSTRAINT fk_notif_alert FOREIGN KEY (alert_id) REFERENCES alerts (alert_id) ON DELETE CASCADE,
    CONSTRAINT fk_notif_user FOREIGN KEY (user_id) REFERENCES users (user_id) ON DELETE CASCADE
);
```

***Purpose**: Stores notification delivery history for triggered price alerts.*
***ER Diagram Check**: ■ Matches*

15. Create Price History Table

```
CREATE TABLE price_history (
    history_id RAW(16) DEFAULT SYS_GUID() PRIMARY KEY,
    symbol VARCHAR2(20) NOT NULL,
    asset_id RAW(16),
    price NUMBER(18, 6) NOT NULL,
    volume NUMBER(24, 2),
    timestamp TIMESTAMP WITH TIME ZONE DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT fk_history_asset FOREIGN KEY (asset_id) REFERENCES assets (asset_id) ON DELETE SET NULL
);
```

***Purpose**: Stores historical timeseries price points for charting and analysis.*
***ER Diagram Check**: ■ Matches*

16. Create News Articles Table

```
CREATE TABLE news_articles (
    article_id RAW(16) DEFAULT SYS_GUID() PRIMARY KEY,
    headline VARCHAR2(500) NOT NULL,
    source VARCHAR2(100) NOT NULL,
    url VARCHAR2(1000) NOT NULL,
    published_at TIMESTAMP WITH TIME ZONE NOT NULL
);
```

***Purpose**: Stores market financial news headlines and metadata.*
***ER Diagram Check**: ■ Matches*

17. Create News Article Assets Junction Table

```
CREATE TABLE news_article_assets (
    article_id RAW(16) NOT NULL,
    asset_id RAW(16) NOT NULL,
    PRIMARY KEY (article_id, asset_id),
    CONSTRAINT fk_naa_article FOREIGN KEY (article_id) REFERENCES news_articles (article_id) ON DELETE CASCADE,
    CONSTRAINT fk_naa_asset FOREIGN KEY (asset_id) REFERENCES assets (asset_id) ON DELETE CASCADE
);
```

***Purpose**: Models the many-to-many relationship linking news articles to mentioned assets.*
***ER Diagram Check**: ■ Matches*

18. Create Price Cache Table

```
CREATE TABLE price_cache (
    cache_id RAW(16) DEFAULT SYS_GUID() PRIMARY KEY,
    symbol VARCHAR2(20) NOT NULL,
    asset_type VARCHAR2(20) NOT NULL,
    price_usd NUMBER(18, 6) NOT NULL,
    high_24h NUMBER(18, 6),
    low_24h NUMBER(18, 6),
    volume_24h NUMBER(24, 2),
    change_24h NUMBER(8, 4),
    fetched_at TIMESTAMP WITH TIME ZONE DEFAULT SYSTIMESTAMP NOT NULL,
    expire_at TIMESTAMP WITH TIME ZONE NOT NULL,
    metadata CLOB,
    CONSTRAINT chk_cache_asset_type CHECK (asset_type IN ('crypto', 'stock')),
    CONSTRAINT chk_cache_json CHECK (metadata IS JSON),
    CONSTRAINT uq_cache_symbol_type UNIQUE (symbol, asset_type)
);
```

***Purpose**: Provides a high-speed price cache table with JSON validation and expiration timestamps.*
***ER Diagram Check**: ■ Matches*

19. Create Audit Logs Table

```
CREATE TABLE audit_logs (
    log_id RAW(16) DEFAULT SYS_GUID() PRIMARY KEY,
    user_id RAW(16) NOT NULL,
    action VARCHAR2(255) NOT NULL,
    timestamp TIMESTAMP WITH TIME ZONE DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT fk_audit_user FOREIGN KEY (user_id) REFERENCES users (user_id) ON DELETE CASCADE
);
```

***Purpose**: Maintains security and compliance audit trails of all user actions.*
***ER Diagram Check**: ■ Matches*

20. Create Relational Performance Indexes

```
CREATE INDEX idx_users_email ON users (email);
CREATE INDEX idx_assets_symbol ON assets (symbol);
CREATE INDEX idx_assets_type ON assets (type);
CREATE INDEX idx_portfolios_user_id ON portfolios (user_id);
CREATE INDEX idx_holdings_portfolio_id ON holdings (portfolio_id);
CREATE INDEX idx_holdings_symbol ON holdings (symbol);
CREATE INDEX idx_tx_portfolio_time ON transactions (portfolio_id, timestamp DESC);
CREATE INDEX idx_tx_user ON transactions (user_id);
CREATE INDEX idx_perf_portfolio_time ON portfolio_performance (portfolio_id, timestamp DESC);
CREATE INDEX idx_watchlist_items_watchlist ON watchlist_items (watchlist_id);
CREATE INDEX idx_alerts_user_enabled ON alerts (user_id, enabled);
CREATE INDEX idx_price_history_sym_time ON price_history (symbol, timestamp DESC);
CREATE INDEX idx_price_cache_expire ON price_cache (expire_at);
CREATE INDEX idx_audit_user_time ON audit_logs (user_id, timestamp DESC);
```

***Purpose**: Optimizes foreign key joins, lookups, and time-series query performance.*
***ER Diagram Check**: ■ Matches*

Part 2: Insert, Update, and Delete (DML) Queries

21. User Registration (Signup) with Output

```
DECLARE
    v_user_id RAW(16);
    v_email VARCHAR2(255);
    v_base_curr VARCHAR2(10);
    v_created_at TIMESTAMP WITH TIME ZONE;
BEGIN
    INSERT INTO users (email, password_hash, base_currency)
    VALUES ('investor@example.com', '$2b$12$e8uq...', 'USD')
    RETURNING user_id, email, base_currency, created_at
    INTO v_user_id, v_email, v_base_curr, v_created_at;

    DBMS_OUTPUT.PUT_LINE('Created User ID: ' || RAWTOHEX(v_user_id) || ' Email: ' || v_email);
END;
/
```

***Purpose**: Inserts a newly registered user account and captures the generated primary key using PL/SQL bind syntax.*
***ER Diagram Check**: ■ Matches*

22. Update User Base Currency Preference

```
UPDATE users
SET base_currency = 'EUR'
WHERE user_id = HEXTORAW('00000000000000000000000000000001');
```

***Purpose**: Updates the display and calculation currency code for a specific user.*
***ER Diagram Check**: ■ Matches*

23. Insert New Catalog Asset

```
DECLARE
    v_asset_id RAW(16);
    v_symbol VARCHAR2(20);
BEGIN
    INSERT INTO assets (symbol, name, type, network, contract_address, ticker_exchange, sector, exchange_id)
    VALUES ('ETH', 'Ethereum', 'crypto', 'Ethereum Mainnet', NULL, NULL, NULL, NULL)
    RETURNING asset_id, symbol INTO v_asset_id, v_symbol;

    DBMS_OUTPUT.PUT_LINE('Cataloged Asset: ' || v_symbol || ' (ID: ' || RAWTOHEX(v_asset_id) || ')');
END;
/
```

***Purpose**: Inserts a new financial asset into the global assets catalog.*
***ER Diagram Check**: ■ Matches*

24. Upsert Ticker Statistics

```
MERGE INTO ticker_stats t
USING (
    SELECT
        HEXTORAW('22222222222222222222222222222222') AS asset_id,
        3250.50 AS price_per_unit,
        18500000000.00 AS volume_24h,
        3310.00 AS high_24h,
        3190.00 AS low_24h,
        SYSTIMESTAMP AS updated_at
    FROM dual
) src
ON (t.asset_id = src.asset_id)
WHEN MATCHED THEN
    UPDATE SET
        t.price_per_unit = src.price_per_unit,
        t.volume_24h     = src.volume_24h,
        t.high_24h       = src.high_24h,
        t.low_24h        = src.low_24h,
        t.updated_at     = src.updated_at
WHEN NOT MATCHED THEN
    INSERT (asset_id, price_per_unit, volume_24h, high_24h, low_24h, updated_at)
    VALUES (src.asset_id, src.price_per_unit, src.volume_24h, src.high_24h, src.low_24h, src.updated_at);
```

***Purpose**: Inserts or updates 24-hour market summary statistics for an asset using Oracle MERGE syntax.*
***ER Diagram Check**: ■ Matches*

25. Create User Portfolio

```
DECLARE
    v_port_id RAW(16);
BEGIN
    INSERT INTO portfolios (user_id, name)
    VALUES (HEXTORAW('00000000000000000000000000000001'), 'Tech Growth Portfolio')
    RETURNING portfolio_id INTO v_port_id;

    DBMS_OUTPUT.PUT_LINE('Created Portfolio ID: ' || RAWTOHEX(v_port_id));
END;
/
```

***Purpose**: Creates a new portfolio entity owned by the specified user.*
***ER Diagram Check**: ■ Matches*

26. Rename User Portfolio

```
UPDATE portfolios
SET name = 'Core Retirement Portfolio'
WHERE portfolio_id = HEXTORAW('11111111111111111111111111111111')
  AND user_id = HEXTORAW('00000000000000000000000000000001');
```

***Purpose**: Renames an existing portfolio while enforcing user-level authorization.*
***ER Diagram Check**: ■ Matches*

27. Delete User Portfolio

```
DELETE FROM portfolios
WHERE portfolio_id = HEXTORAW('11111111111111111111111111111111')
  AND user_id = HEXTORAW('00000000000000000000000000000001');
```

***Purpose**: Deletes a portfolio and cascades deletion to all embedded holdings and performance records.*
***ER Diagram Check**: ■ Matches*

28. Upsert Position with Weighted Average Cost Basis

```
MERGE INTO holdings h
USING (
    SELECT
        HEXTORAW('11111111111111111111111111111111') AS portfolio_id,
        HEXTORAW('22222222222222222222222222222222') AS asset_id,
        'BTC' AS symbol,
        'Bitcoin' AS name,
        'crypto' AS type,
        0.75 AS quantity,
        62000.00 AS average_buy_price
    FROM dual
) src
ON (h.portfolio_id = src.portfolio_id AND h.symbol = src.symbol)
WHEN MATCHED THEN
    UPDATE SET
        h.quantity = h.quantity + src.quantity,
        h.average_buy_price = ((h.quantity * h.average_buy_price) + (src.quantity * src.average_buy_price)) / (h.quantity + src.quantity
        h.updated_at = SYSTIMESTAMP
WHEN NOT MATCHED THEN
    INSERT (holding_id, portfolio_id, asset_id, symbol, name, type, quantity, average_buy_price, updated_at)
    VALUES (SYS_GUID(), src.portfolio_id, src.asset_id, src.symbol, src.name, src.type, src.quantity, src.average_buy_price, SYSTIMESTAM
```

***Purpose**: Inserts a new holding or aggregates quantity and recalculates the weighted average purchase price on conflict.*
***ER Diagram Check**: ■ Matches*

29. Update Holding Position

```
UPDATE holdings
SET quantity = 1.25, average_buy_price = 61500.00, updated_at = SYSTIMESTAMP
WHERE holding_id = HEXTORAW('33333333333333333333333333333333')
  AND portfolio_id = HEXTORAW('11111111111111111111111111111111');
```

***Purpose**: Manually modifies the quantity and buy price of a specific holding within a portfolio.*
***ER Diagram Check**: ■ Matches*

30. Delete Holding Position

```
DELETE FROM holdings
WHERE holding_id = HEXTORAW('33333333333333333333333333333333')
  AND portfolio_id = HEXTORAW('11111111111111111111111111111111');
```

***Purpose**: Removes an asset position from a portfolio.*
***ER Diagram Check**: ■ Matches*

31. Record Trade Execution Log

```
DECLARE
    v_tx_id RAW(16);
BEGIN
    INSERT INTO transactions (user_id, portfolio_id, asset_id, symbol, type, quantity, price_at_transaction, network_hash)
    VALUES (
        HEXTORAW('00000000000000000000000000000001'),
        HEXTORAW('11111111111111111111111111111111'),
        HEXTORAW('22222222222222222222222222222222'),
        'NVDA', 'BUY', 20.0, 125.50, NULL
    )
    RETURNING transaction_id INTO v_tx_id;

    DBMS_OUTPUT.PUT_LINE('Executed Transaction ID: ' || RAWTOHEX(v_tx_id));
END;
/
```

***Purpose**: Inserts an immutable transaction record for an executed trade or asset transfer.*
***ER Diagram Check**: ■ Matches*

32. Record Periodic Portfolio Performance Snapshot

```
INSERT INTO portfolio_performance (portfolio_id, total_value, total_profit_loss, total_profit_loss_pct, timestamp)
VALUES (HEXTORAW('11111111111111111111111111111111'), 128500.00, 18500.00, 16.8182, SYSTIMESTAMP);
```

***Purpose**: Saves a historical snapshot of portfolio valuation and aggregate profit/loss.*
***ER Diagram Check**: ■ Matches*

33. Create User Watchlist

```
DECLARE
    v_wl_id RAW(16);
BEGIN
    INSERT INTO watchlists (user_id, name)
    VALUES (HEXTORAW('00000000000000000000000000000001'), 'Crypto Blue Chips')
    RETURNING watchlist_id INTO v_wl_id;

    DBMS_OUTPUT.PUT_LINE('Created Watchlist ID: ' || RAWTOHEX(v_wl_id));
END;
/
```

***Purpose**: Creates a new custom watchlist container for a user.*
***ER Diagram Check**: ■ Matches*

34. Add Symbol to Watchlist (Idempotent)

```
MERGE INTO watchlist_items w
USING (
    SELECT
        HEXTORAW('55555555555555555555555555555555') AS watchlist_id,
        'SOL' AS symbol,
        HEXTORAW('66666666666666666666666666666666') AS asset_id
    FROM dual
) src
ON (w.watchlist_id = src.watchlist_id AND w.symbol = src.symbol)
WHEN NOT MATCHED THEN
    INSERT (item_id, watchlist_id, symbol, asset_id, added_at)
    VALUES (SYS_GUID(), src.watchlist_id, src.symbol, src.asset_id, SYSTIMESTAMP);
```

***Purpose**: Adds an asset symbol to a specified user watchlist if not already present.*
***ER Diagram Check**: ■ Matches*

35. Remove Symbol from Watchlist

```
DELETE FROM watchlist_items
WHERE watchlist_id = HEXTORAW('55555555555555555555555555555555') AND symbol = 'SOL';
```

***Purpose**: Deletes a tracked asset symbol from a user's watchlist.*
***ER Diagram Check**: ■ Matches*

36. Create Price Alert Rule

```
DECLARE
    v_alert_id RAW(16);
BEGIN
    INSERT INTO alerts (user_id, symbol, asset_id, threshold, direction, enabled)
    VALUES (
        HEXTORAW('00000000000000000000000000000001'),
        'BTC',
        HEXTORAW('22222222222222222222222222222222'),
        70000.00, 'ABOVE', 1
    )
    RETURNING alert_id INTO v_alert_id;

    DBMS_OUTPUT.PUT_LINE('Created Price Alert ID: ' || RAWTOHEX(v_alert_id));
END;
/
```

***Purpose**: Inserts a new price monitoring rule for a target asset and threshold.*
***ER Diagram Check**: ■ Matches*

37. Update Alert Status on Trigger

```
UPDATE alerts
SET triggered_at = SYSTIMESTAMP, enabled = 0
WHERE alert_id = HEXTORAW('44444444444444444444444444444444');
```

***Purpose**: Disables an alert and marks its triggered timestamp once criteria are met.*
***ER Diagram Check**: ■ Matches*

38. Delete Price Alert

```
DELETE FROM alerts
WHERE alert_id = HEXTORAW('44444444444444444444444444444444')
  AND user_id = HEXTORAW('00000000000000000000000000000001');
```

***Purpose**: Deletes a user price alert rule.*
***ER Diagram Check**: ■ Matches*

39. Insert Notification Log

```
DECLARE
    v_notif_id RAW(16);
BEGIN
    INSERT INTO notifications (alert_id, user_id, channel, status)
    VALUES (
        HEXTORAW('44444444444444444444444444444444'),
        HEXTORAW('00000000000000000000000000000001'),
        'EMAIL', 'SENT'
    )
    RETURNING notification_id INTO v_notif_id;

    DBMS_OUTPUT.PUT_LINE('Dispatched Notification ID: ' || RAWTOHEX(v_notif_id));
END;
/
```

***Purpose**: Records an audit log entry for a dispatched user notification.*
***ER Diagram Check**: ■ Matches*

40. Insert Price History Point

```
INSERT INTO price_history (symbol, asset_id, price, volume, timestamp)
VALUES ('BTC', HEXTORAW('22222222222222222222222222222222'), 64320.00, 24500000000.00, SYSTIMESTAMP);
```

***Purpose**: Inserts a historical price timestamp record for asset charting.*
***ER Diagram Check**: ■ Matches*

41. Insert News Article

```
INSERT INTO news_articles (article_id, headline, source, url, published_at)
VALUES (
    HEXTORAW('77777777777777777777777777777777'),
    'Fed Signals Steady Interest Rates', 'Reuters',
    'https://reuters.com/article1', SYSTIMESTAMP
);
```

***Purpose**: Inserts a new financial news publication record.*
***ER Diagram Check**: ■ Matches*

42. Link News Article to Asset (Idempotent)

```
MERGE INTO news_article_assets naa
USING (
    SELECT
        HEXTORAW('77777777777777777777777777777777') AS article_id,
        HEXTORAW('22222222222222222222222222222222') AS asset_id
    FROM dual
) src
ON (naa.article_id = src.article_id AND naa.asset_id = src.asset_id)
WHEN NOT MATCHED THEN
    INSERT (article_id, asset_id)
    VALUES (src.article_id, src.asset_id);
```

***Purpose**: Maps a news article to an associated asset ticker without duplicating rows.*
***ER Diagram Check**: ■ Matches*

43. Upsert Price Cache Entry

```
MERGE INTO price_cache pc
USING (
    SELECT
        'BTC' AS symbol,
        'crypto' AS asset_type,
        64200.00 AS price_usd,
        65000.00 AS high_24h,
        63800.00 AS low_24h,
        29000000000.00 AS volume_24h,
        2.45 AS change_24h,
        SYSTIMESTAMP AS fetched_at,
        SYSTIMESTAMP + INTERVAL '5' MINUTE AS expire_at,
        '{"provider": "coingecko"}' AS metadata
    FROM dual
) src
ON (pc.symbol = src.symbol AND pc.asset_type = src.asset_type)
WHEN MATCHED THEN
    UPDATE SET
        pc.price_usd   = src.price_usd,
        pc.high_24h    = src.high_24h,
        pc.low_24h     = src.low_24h,
        pc.volume_24h  = src.volume_24h,
        pc.change_24h  = src.change_24h,
        pc.fetched_at  = src.fetched_at,
        pc.expire_at   = src.expire_at,

        pc.metadata    = src.metadata
WHEN NOT MATCHED THEN
    INSERT (cache_id, symbol, asset_type, price_usd, high_24h, low_24h, volume_24h, change_24h, fetched_at, expire_at, metadata)
    VALUES (SYS_GUID(), src.symbol, src.asset_type, src.price_usd, src.high_24h, src.low_24h, src.volume_24h, src.change_24h, src.fetch
```

***Purpose**: Updates or inserts external API quote results with a 5-minute expiration time.*
***ER Diagram Check**: ■ Matches*

44. Record User Audit Log

```
INSERT INTO audit_logs (user_id, action, timestamp)
VALUES (HEXTORAW('00000000000000000000000000000001'), 'USER_LOGIN', SYSTIMESTAMP);
```

***Purpose**: Logs user security and configuration events for auditing.*
***ER Diagram Check**: ■ Matches*

Part 3: SELECT, Analytical, and Reporting Queries

45. User Authentication Credentials Lookup

```
SELECT user_id, email, password_hash, base_currency, created_at
FROM users
WHERE email = 'investor@example.com';
```

***Purpose**: Retrieves user credentials and security hash for password verification during login.*
***ER Diagram Check**: ■ Matches*

46. Fetch Current User Profile

```
SELECT u.user_id, u.email, u.base_currency, u.created_at, cr.rate_to_usd
FROM users u
LEFT JOIN currency_rates cr ON u.base_currency = cr.currency_code
WHERE u.user_id = HEXTORAW('00000000000000000000000000000001');
```

***Purpose**: Fetches the authenticated user profile along with currency exchange rate details.*
***ER Diagram Check**: ■ Matches*

47. List User Portfolios with Total Position Counts

```
SELECT
    p.portfolio_id,
    p.name,
    p.created_at,
    COUNT(h.holding_id) AS total_positions
FROM portfolios p
LEFT JOIN holdings h ON p.portfolio_id = h.portfolio_id
WHERE p.user_id = HEXTORAW('00000000000000000000000000000001')
GROUP BY p.portfolio_id, p.name, p.created_at
ORDER BY p.created_at DESC;
```

***Purpose**: Returns all portfolios owned by a user along with the count of active holdings.*
***ER Diagram Check**: ■ Matches*

48. Retrieve Single Portfolio by ID

```
SELECT portfolio_id, user_id, name, created_at
FROM portfolios
WHERE portfolio_id = HEXTORAW('11111111111111111111111111111111')
  AND user_id = HEXTORAW('00000000000000000000000000000001');
```

***Purpose**: Retrieves a specific portfolio while enforcing user ownership boundaries.*
***ER Diagram Check**: ■ Matches*

49. List All Holdings in a Specific Portfolio

```
SELECT
    holding_id,
    portfolio_id,
    symbol,
    name,
    type,
    quantity,
    average_buy_price,
    updated_at
FROM holdings
WHERE portfolio_id = HEXTORAW('11111111111111111111111111111111')
ORDER BY symbol ASC;
```

***Purpose**: Fetches all holding positions belonging to a portfolio.*
***ER Diagram Check**: ■ Matches*

50. Get Specific Holding Position by ID

```
SELECT h.holding_id, h.portfolio_id, h.symbol, h.name, h.type, h.quantity, h.average_buy_price, h.updated_at
FROM holdings h
JOIN portfolios p ON h.portfolio_id = p.portfolio_id
WHERE h.holding_id = HEXTORAW('33333333333333333333333333333333')
  AND p.user_id = HEXTORAW('00000000000000000000000000000001');
```

***Purpose**: Retrieves single holding details ensuring the user owns the parent portfolio.*
***ER Diagram Check**: ■ Matches*

51. Real-Time Holdings Valuation with Unrealized Gain/Loss

```
SELECT
    h.holding_id,
    h.symbol,
    h.name,
    h.type,
    h.quantity,
    h.average_buy_price,
    (h.quantity * h.average_buy_price) AS total_invested,
    NVL(pc.price_usd, NVL(ts.price_per_unit, 0)) AS current_price,
    (h.quantity * NVL(pc.price_usd, NVL(ts.price_per_unit, 0))) AS current_value,
    ((h.quantity * NVL(pc.price_usd, NVL(ts.price_per_unit, 0))) - (h.quantity * h.average_buy_price)) AS gain_loss,
    CASE
        WHEN (h.quantity * h.average_buy_price) > 0
        THEN (((h.quantity * NVL(pc.price_usd, NVL(ts.price_per_unit, 0))) - (h.quantity * h.average_buy_price)) / (h.quantity * h.avera
        ELSE 0
    END AS gain_loss_pct
FROM holdings h
LEFT JOIN assets a ON h.symbol = a.symbol
LEFT JOIN ticker_stats ts ON a.asset_id = ts.asset_id
LEFT JOIN price_cache pc ON h.symbol = pc.symbol AND pc.expire_at > SYSTIMESTAMP
WHERE h.portfolio_id = HEXTORAW('11111111111111111111111111111111')
ORDER BY current_value DESC;
```

***Purpose**: Joins positions with cached and catalog prices to calculate current position value, unrealized profit/loss, and ROI %.*
***ER Diagram Check**: ■ Matches*

52. Portfolio Total Valuation & Base Currency Conversion

```
SELECT
    p.portfolio_id,
    p.name AS portfolio_name,
    u.base_currency,
    cr.rate_to_usd,
    SUM(h.quantity * h.average_buy_price) AS total_cost_usd,
    SUM(h.quantity * NVL(pc.price_usd, NVL(ts.price_per_unit, 0))) AS total_market_value_usd,
    (SUM(h.quantity * NVL(pc.price_usd, NVL(ts.price_per_unit, 0))) - SUM(h.quantity * h.average_buy_price)) AS total_unrealized_pnl_usd,
    CASE
        WHEN SUM(h.quantity * h.average_buy_price) > 0
        THEN ((SUM(h.quantity * NVL(pc.price_usd, NVL(ts.price_per_unit, 0))) - SUM(h.quantity * h.average_buy_price)) / SUM(h.quantity * h.average_buy_price)) * 100
        ELSE 0
    END AS total_unrealized_pnl_pct,
    SUM(h.quantity * NVL(pc.price_usd, NVL(ts.price_per_unit, 0))) * cr.rate_to_usd AS total_value_base_currency
FROM portfolios p
JOIN users u ON p.user_id = u.user_id
JOIN currency_rates cr ON u.base_currency = cr.currency_code
JOIN holdings h ON p.portfolio_id = h.portfolio_id
LEFT JOIN assets a ON h.symbol = a.symbol
LEFT JOIN ticker_stats ts ON a.asset_id = ts.asset_id
LEFT JOIN price_cache pc ON h.symbol = pc.symbol AND pc.expire_at > SYSTIMESTAMP
WHERE p.portfolio_id = HEXTORAW('11111111111111111111111111111111')
GROUP BY p.portfolio_id, p.name, u.base_currency, cr.rate_to_usd;
```

***Purpose**: Computes aggregate portfolio cost, market value, PnL, and converted value in user's base currency.*

***ER Diagram Check**: ■ Matches*

53. Asset Class Allocation Breakdown (Crypto vs Stock)

```
WITH PositionValues AS (
    SELECT
        h.type,
        SUM(h.quantity * NVL(pc.price_usd, NVL(ts.price_per_unit, 0))) AS class_value
    FROM holdings h
    LEFT JOIN assets a ON h.symbol = a.symbol
    LEFT JOIN ticker_stats ts ON a.asset_id = ts.asset_id
    LEFT JOIN price_cache pc ON h.symbol = pc.symbol AND pc.expire_at > SYSTIMESTAMP
    WHERE h.portfolio_id = HEXTORAW('11111111111111111111111111111111')
    GROUP BY h.type
),
TotalValue AS (
    SELECT SUM(class_value) AS grand_total FROM PositionValues
)
SELECT
    pv.type AS asset_class,
    pv.class_value,
    ROUND((pv.class_value / NULLIF(tv.grand_total, 0)) * 100, 2) AS allocation_percentage
FROM PositionValues pv, TotalValue tv;
```

***Purpose**: Calculates portfolio asset distribution percentage between crypto and equity asset classes.*

***ER Diagram Check**: ■ Matches*

54. Individual Asset Allocation Weighting

```
WITH PortfolioValuation AS (
    SELECT
        h.holding_id,
        h.symbol,
        h.type,
        (h.quantity * NVL(pc.price_usd, NVL(ts.price_per_unit, 0))) AS market_value
    FROM holdings h
    LEFT JOIN assets a ON h.symbol = a.symbol
    LEFT JOIN ticker_stats ts ON a.asset_id = ts.asset_id
    LEFT JOIN price_cache pc ON h.symbol = pc.symbol AND pc.expire_at > SYSTIMESTAMP
    WHERE h.portfolio_id = HEXTORAW('11111111111111111111111111111111')
),
TotalValuation AS (
    SELECT SUM(market_value) AS total_market_value FROM PortfolioValuation
)
SELECT
    pv.symbol,
    pv.type,
    pv.market_value,
    ROUND((pv.market_value / NULLIF(tv.total_market_value, 0)) * 100, 2) AS allocation_percentage
FROM PortfolioValuation pv, TotalValuation tv
ORDER BY pv.market_value DESC;
```

***Purpose**: Computes each asset's percentage contribution to the total portfolio value.*
***ER Diagram Check**: ■ Matches*

55. Fetch Paginated Portfolio Transaction History

```
SELECT
    t.transaction_id,
    t.symbol,
    t.type,
    t.quantity,
    t.price_at_transaction,
    (t.quantity * t.price_at_transaction) AS total_amount,
    t.network_hash,
    t.timestamp
FROM transactions t
WHERE t.portfolio_id = HEXTORAW('11111111111111111111111111111111')
ORDER BY t.timestamp DESC
OFFSET 0 ROWS FETCH NEXT 50 ROWS ONLY;
```

***Purpose**: Returns paginated trade execution history for a portfolio sorted chronologically descending.*
***ER Diagram Check**: ■ Matches*

56. List User Watchlists with Live Market Prices

```
SELECT
    w.watchlist_id,
    w.name AS watchlist_name,
    wi.item_id,
    wi.symbol,
    wi.added_at,
    NVL(pc.price_usd, NVL(ts.price_per_unit, 0)) AS current_price,
    pc.change_24h
FROM watchlists w
JOIN watchlist_items wi ON w.watchlist_id = wi.watchlist_id
LEFT JOIN assets a ON wi.symbol = a.symbol
LEFT JOIN ticker_stats ts ON a.asset_id = ts.asset_id
LEFT JOIN price_cache pc ON wi.symbol = pc.symbol
WHERE w.user_id = HEXTORAW('00000000000000000000000000000001')
ORDER BY w.name ASC, wi.added_at DESC;
```

***Purpose**: Retrieves all user watchlists and items combined with 24-hour price and change data.*
***ER Diagram Check**: ■ Matches*

57. Active Price Alerts Trigger Evaluation

```
SELECT
    al.alert_id,
    al.user_id,
    al.symbol,
    al.threshold,
    al.direction,
    NVL(pc.price_usd, ts.price_per_unit) AS current_price,
    u.email
FROM alerts al
JOIN users u ON al.user_id = u.user_id
LEFT JOIN assets a ON al.symbol = a.symbol
LEFT JOIN ticker_stats ts ON a.asset_id = ts.asset_id
LEFT JOIN price_cache pc ON al.symbol = pc.symbol
WHERE al.enabled = 1
  AND (
      (al.direction = 'ABOVE' AND NVL(pc.price_usd, ts.price_per_unit) >= al.threshold)
      OR
      (al.direction = 'BELOW' AND NVL(pc.price_usd, ts.price_per_unit) <= al.threshold)
  );
```

***Purpose**: Evaluates current prices against enabled threshold conditions to determine alerts requiring dispatch.*
***ER Diagram Check**: ■ Matches*

58. Top Performing Holdings by Unrealized Return (%)

```
SELECT
    h.symbol,
    h.name,
    h.type,
    (h.quantity * NVL(pc.price_usd, NVL(ts.price_per_unit, 0))) AS current_value,
    (((NVL(pc.price_usd, NVL(ts.price_per_unit, 0)) - h.average_buy_price) / NULLIF(h.average_buy_price, 0)) * 100) AS return_pct
FROM holdings h
JOIN portfolios p ON h.portfolio_id = p.portfolio_id
LEFT JOIN assets a ON h.symbol = a.symbol
LEFT JOIN ticker_stats ts ON a.asset_id = ts.asset_id
LEFT JOIN price_cache pc ON h.symbol = pc.symbol AND pc.expire_at > SYSTIMESTAMP
WHERE p.user_id = HEXTORAW('00000000000000000000000000000001')
ORDER BY return_pct DESC
FETCH FIRST 5 ROWS ONLY;
```

***Purpose**: Ranks top 5 highest returning holdings across all portfolios of a user.*
***ER Diagram Check**: ■ Matches*

59. 24-Hour Historical Performance Snapshot Series

```
SELECT
    pp.performance_id,
    pp.total_value,
    pp.total_profit_loss,
    pp.total_profit_loss_pct,
    pp.timestamp
FROM portfolio_performance pp
WHERE pp.portfolio_id = HEXTORAW('11111111111111111111111111111111')
  AND pp.timestamp >= SYSTIMESTAMP - INTERVAL '24' HOUR
ORDER BY pp.timestamp ASC;
```

***Purpose**: Retrieves 24-hour timeseries data points for plotting portfolio valuation charts.*
***ER Diagram Check**: ■ Matches*

60. Asset News Feed for User Portfolio Holdings

```
SELECT DISTINCT
    na.article_id,
    na.headline,
    na.source,
    na.url,
    na.published_at,
    a.symbol
FROM news_articles na
JOIN news_article_assets naa ON na.article_id = naa.article_id
JOIN assets a ON naa.asset_id = a.asset_id
JOIN holdings h ON a.symbol = h.symbol
JOIN portfolios p ON h.portfolio_id = p.portfolio_id
WHERE p.user_id = HEXTORAW('00000000000000000000000000000001')
ORDER BY na.published_at DESC
FETCH FIRST 20 ROWS ONLY;
```

***Purpose**: Fetches relevant market news articles filtered by assets actively held in user portfolios.*
***ER Diagram Check**: ■ Matches*

61. User Activity & Audit Log Trail

```
SELECT log_id, user_id, action, timestamp
FROM audit_logs
WHERE user_id = HEXTORAW('00000000000000000000000000000001')
ORDER BY timestamp DESC
FETCH FIRST 100 ROWS ONLY;
```

***Purpose**: Retrieves recent security and transactional activity audit records for compliance.*
***ER Diagram Check**: ■ Matches*

Part 4: PL/SQL Procedures, Functions & Triggers

62. Auto-Update Holdings Modification Timestamp Trigger

```
CREATE OR REPLACE TRIGGER trg_update_holdings_timestamp
BEFORE UPDATE ON holdings
FOR EACH ROW
BEGIN
    :NEW.updated_at := SYSTIMESTAMP;
END;
/
```

***Purpose**: Automatically synchronizes the `updated_at` column with `SYSTIMESTAMP` whenever a holding record is modified.*
***ER Diagram Check**: ■ Matches*

63. Auto-Insert Transaction Activity into Audit Logs Trigger

```
CREATE OR REPLACE TRIGGER trg_log_transaction_activity
AFTER INSERT ON transactions
FOR EACH ROW
BEGIN
    INSERT INTO audit_logs (
        log_id,
        user_id,
        action,
        timestamp
    ) VALUES (
        SYS_GUID(),
        :NEW.user_id,
        'TRADE_EXECUTION: ' || :NEW.type || ' ' || TO_CHAR(:NEW.quantity) || ' ' || :NEW.symbol || ' @ ' || TO_CHAR(:NEW.price_at_transa
        COALESCE(:NEW.timestamp, SYSTIMESTAMP)
    );
END;
/
```

***Purpose**: Automatically writes an immutable audit record to `audit_logs` whenever a new trade or asset transfer is inserted.*
***ER Diagram Check**: ■ Matches*

64. Weighted-Average-Cost Holding Position Upsert

```
CREATE OR REPLACE FUNCTION upsert_holding(
    p_portfolio_id IN RAW,
    p_asset_id     IN RAW,
    p_symbol       IN VARCHAR2,
    p_name         IN VARCHAR2,
    p_type         IN VARCHAR2,
    p_quantity     IN NUMBER,
    p_buy_price    IN NUMBER
)
RETURN RAW
IS
    v_holding_id RAW(16);
BEGIN
    -- Input validation
    IF p_quantity <= 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Holding quantity must be strictly positive (received: ' || TO_CHAR(p_quantity) || ')');
    END IF;

    IF p_buy_price < 0 THEN
        RAISE_APPLICATION_ERROR(-20002, 'Holding purchase price cannot be negative (received: ' || TO_CHAR(p_buy_price) || ')');
    END IF;

    IF p_type NOT IN ('crypto', 'stock') THEN
        RAISE_APPLICATION_ERROR(-20003, 'Invalid asset type "' || p_type || '". Must be "crypto" or "stock"');
    END IF;

    -- Perform atomic MERGE with weighted average cost recalculation
    MERGE INTO holdings h
    USING (
        SELECT
            p_portfolio_id AS portfolio_id,
            p_asset_id     AS asset_id,
            p_symbol       AS symbol,
            p_name         AS name,
            p_type         AS type,
            p_quantity     AS quantity,
            p_buy_price    AS buy_price
        FROM dual
    ) src
    ON (h.portfolio_id = src.portfolio_id AND h.symbol = src.symbol)
    WHEN MATCHED THEN
        UPDATE SET
            h.quantity = h.quantity + src.quantity,
            h.average_buy_price = ((h.quantity * h.average_buy_price) + (src.quantity * src.buy_price)) / (h.quantity + src.quantity),
            h.updated_at = SYSTIMESTAMP
    WHEN NOT MATCHED THEN
        INSERT (holding_id, portfolio_id, asset_id, symbol, name, type, quantity, average_buy_price, updated_at)
        VALUES (SYS_GUID(), src.portfolio_id, src.asset_id, src.symbol, src.name, src.type, src.quantity, src.buy_price, SYSTIMESTAMP);

    SELECT holding_id INTO v_holding_id
    FROM holdings
    WHERE portfolio_id = p_portfolio_id AND symbol = p_symbol;

    RETURN v_holding_id;
EXCEPTION
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20004, 'Failed to upsert holding for symbol ' || p_symbol || ': ' || SQLERRM);
END upsert_holding;
/
```

***Purpose**: Safely upserts an asset holding in a portfolio, recomputing its weighted-average cost basis and enforcing validation.*
***ER Diagram Check**: ■ Matches*

65. Atomic Transaction Recording and Holdings Synchronization

```
CREATE OR REPLACE FUNCTION record_transaction_and_update_holding(
    p_user_id      IN RAW,
    p_portfolio_id IN RAW,
    p_asset_id     IN RAW,
    p_symbol       IN VARCHAR2,
    p_type         IN VARCHAR2,
    p_quantity     IN NUMBER,
    p_price        IN NUMBER,
    p_network_hash IN VARCHAR2 DEFAULT NULL,
    p_asset_name   IN VARCHAR2 DEFAULT NULL,
    p_asset_type   IN VARCHAR2 DEFAULT NULL
)
RETURN RAW
IS
    v_tx_id          RAW(16);
    v_current_qty    NUMBER(24, 8);
    v_current_avg    NUMBER(18, 6);
    v_resolved_name  VARCHAR2(150);
    v_resolved_type  VARCHAR2(20);
BEGIN
    -- Validate inputs
    IF p_quantity <= 0 THEN
        RAISE_APPLICATION_ERROR(-20010, 'Transaction quantity must be strictly greater than 0');
    END IF;

    IF p_price < 0 THEN
        RAISE_APPLICATION_ERROR(-20011, 'Transaction price cannot be negative');
    END IF;

    IF p_type NOT IN ('BUY', 'SELL', 'TRANSFER_IN', 'TRANSFER_OUT') THEN
        RAISE_APPLICATION_ERROR(-20012, 'Invalid transaction type: ' || p_type || '. Expected BUY, SELL, TRANSFER_IN, or TRANSFER_OUT');
    END IF;

    -- Resolve asset metadata if not supplied
    IF p_asset_name IS NULL OR p_asset_type IS NULL THEN
        BEGIN
            SELECT name, type INTO v_resolved_name, v_resolved_type
            FROM assets
            WHERE symbol = p_symbol
              AND ROWNUM = 1;
        EXCEPTION
            WHEN NO_DATA_FOUND THEN
                v_resolved_name := p_symbol;
                v_resolved_type := 'crypto';
        END;
        v_resolved_name := COALESCE(v_resolved_name, p_symbol);
        v_resolved_type := COALESCE(v_resolved_type, 'crypto');
    ELSE
        v_resolved_name := p_asset_name;
        v_resolved_type := p_asset_type;
```

```
    END IF;

    -- Handle Holdings Adjustment
    IF p_type IN ('BUY', 'TRANSFER_IN') THEN
        MERGE INTO holdings h
        USING (
            SELECT
                p_portfolio_id  AS portfolio_id,
                p_asset_id      AS asset_id,
                p_symbol        AS symbol,
                v_resolved_name AS name,
                v_resolved_type AS type,
                p_quantity      AS quantity,
                p_price         AS price
            FROM dual
        ) src
        ON (h.portfolio_id = src.portfolio_id AND h.symbol = src.symbol)
        WHEN MATCHED THEN
            UPDATE SET
                h.quantity = h.quantity + src.quantity,
                h.average_buy_price = CASE
                    WHEN (h.quantity + src.quantity) > 0
                    THEN ((h.quantity * h.average_buy_price) + (src.quantity * src.price)) / (h.quantity + src.quantity)
                    ELSE h.average_buy_price
                END,
                h.updated_at = SYSTIMESTAMP
        WHEN NOT MATCHED THEN
            INSERT (holding_id, portfolio_id, asset_id, symbol, name, type, quantity, average_buy_price, updated_at)
            VALUES (SYS_GUID(), src.portfolio_id, src.asset_id, src.symbol, src.name, src.type, src.quantity, src.price, SYSTIMESTAMP);

    ELSIF p_type IN ('SELL', 'TRANSFER_OUT') THEN
        BEGIN
            SELECT quantity, average_buy_price INTO v_current_qty, v_current_avg
            FROM holdings
            WHERE portfolio_id = p_portfolio_id AND symbol = p_symbol;
        EXCEPTION
            WHEN NO_DATA_FOUND THEN
                RAISE_APPLICATION_ERROR(-20013, 'Cannot execute ' || p_type || ': No existing position for symbol ' || p_symbol || ' in
        END;

        IF v_current_qty < p_quantity THEN
            RAISE_APPLICATION_ERROR(-20014, 'Insufficient position balance: attempted to ' || p_type || ' ' || TO_CHAR(p_quantity) || '
        ELSIF v_current_qty = p_quantity THEN
            DELETE FROM holdings
            WHERE portfolio_id = p_portfolio_id AND symbol = p_symbol;
        ELSE
            UPDATE holdings
            SET quantity = quantity - p_quantity,
                updated_at = SYSTIMESTAMP
            WHERE portfolio_id = p_portfolio_id AND symbol = p_symbol;
```

```
        END IF;
    END IF;

    -- Insert Transaction Ledger Record
    v_tx_id := SYS_GUID();
    INSERT INTO transactions (
        transaction_id,
        user_id,
        portfolio_id,
        asset_id,
        symbol,
        type,
        quantity,
        price_at_transaction,
        network_hash,
        timestamp
    ) VALUES (
        v_tx_id,
        p_user_id,
        p_portfolio_id,
        p_asset_id,
        p_symbol,
        p_type,
        p_quantity,
        p_price,
        p_network_hash,
        SYSTIMESTAMP
    );

    RETURN v_tx_id;
EXCEPTION
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20015, 'Transaction execution failed and state rolled back: ' || SQLERRM);
END record_transaction_and_update_holding;
/
```

***Purpose**: Atomically executes a trade order by validating position balances, updating or deleting rows in `holdings` with weighted-average cost adjustments, and inserting the ledger entry.*
***ER Diagram Check**: ■ Matches*

66. Active Price Alerts Evaluation and Notification Dispatch

```
CREATE OR REPLACE FUNCTION check_price_alerts
RETURN NUMBER
IS
    v_triggered_count NUMBER := 0;
    v_current_price   NUMBER(18, 6);
BEGIN
    FOR v_alert IN (
        SELECT
            al.alert_id,
            al.user_id,
            al.symbol,
            al.threshold,
            al.direction,
            COALESCE(pc.price_usd, ts.price_per_unit) AS current_price
        FROM alerts al
        LEFT JOIN assets a ON al.symbol = a.symbol
        LEFT JOIN ticker_stats ts ON a.asset_id = ts.asset_id
        LEFT JOIN price_cache pc ON al.symbol = pc.symbol
        WHERE al.enabled = 1
    ) LOOP
        BEGIN
            v_current_price := v_alert.current_price;

            IF v_current_price IS NOT NULL THEN
                IF (v_alert.direction = 'ABOVE' AND v_current_price >= v_alert.threshold) OR
                   (v_alert.direction = 'BELOW' AND v_current_price <= v_alert.threshold) THEN

                    -- Insert Notification Record
                    INSERT INTO notifications (
                        notification_id,
                        alert_id,
                        user_id,
                        channel,
                        status,
                        sent_at
                    ) VALUES (
                        SYS_GUID(),
                        v_alert.alert_id,
                        v_alert.user_id,
                        'IN_APP',
                        'PENDING',
                        SYSTIMESTAMP
                    );

                    -- Deactivate Alert & Record Trigger Timestamp
                    UPDATE alerts
                    SET enabled = 0,
                        triggered_at = SYSTIMESTAMP
                    WHERE alert_id = v_alert.alert_id;

                    v_triggered_count := v_triggered_count + 1;
                END IF;
            END IF;
        EXCEPTION
            WHEN OTHERS THEN
                -- Isolate failure to single alert so entire evaluation loop completes
                DBMS_OUTPUT.PUT_LINE('WARNING: Error evaluating alert ID ' || RAWTOHEX(v_alert.alert_id) || ' for symbol ' || v_alert.sy
        END;
    END LOOP;

    RETURN v_triggered_count;
END check_price_alerts;
/
```

***Purpose**: Iterates through all active price alerts, compares real-time asset market prices against user thresholds, dispatches notifications, and deactivates triggered alerts.*
***ER Diagram Check**: ■ Matches*

67. Live Portfolio Valuation (Returns SYS_REFCURSOR)

```
CREATE OR REPLACE FUNCTION get_portfolio_valuation(p_portfolio_id IN RAW)
RETURN SYS_REFCURSOR
IS
    p_cursor SYS_REFCURSOR;
BEGIN
    OPEN p_cursor FOR
        SELECT
            h.holding_id,
            h.symbol,
            h.name,
            h.type,
            h.quantity,
            h.average_buy_price,
            COALESCE(pc.price_usd, ts.price_per_unit, 0.0) AS current_price,
            (h.quantity * h.average_buy_price) AS cost_basis,
            (h.quantity * COALESCE(pc.price_usd, ts.price_per_unit, 0.0)) AS market_value,
            ((h.quantity * COALESCE(pc.price_usd, ts.price_per_unit, 0.0)) - (h.quantity * h.average_buy_price)) AS unrealized_pnl,
            CASE
                WHEN h.average_buy_price > 0
                THEN (((COALESCE(pc.price_usd, ts.price_per_unit, 0.0) - h.average_buy_price) / h.average_buy_price) * 100.0)
                ELSE 0.0
            END AS unrealized_pnl_pct,
            u.base_currency AS portfolio_base_currency,
            COALESCE(cr.rate_to_usd, 1.0) AS currency_rate_to_usd,
            ((h.quantity * COALESCE(pc.price_usd, ts.price_per_unit, 0.0)) / COALESCE(cr.rate_to_usd, 1.0)) AS market_value_base_currenc

        FROM holdings h
        JOIN portfolios p ON h.portfolio_id = p.portfolio_id
        JOIN users u ON p.user_id = u.user_id
        LEFT JOIN currency_rates cr ON u.base_currency = cr.currency_code
        LEFT JOIN assets a ON h.symbol = a.symbol
        LEFT JOIN ticker_stats ts ON a.asset_id = ts.asset_id
        LEFT JOIN price_cache pc ON h.symbol = pc.symbol AND pc.expire_at > SYSTIMESTAMP
        WHERE h.portfolio_id = p_portfolio_id
        ORDER BY market_value DESC;

    RETURN p_cursor;
END get_portfolio_valuation;
/
```

***Purpose**: Computes comprehensive real-time position valuations, cost basis, unrealized gain/loss amounts, and foreign currency conversion, returning the dataset as an Oracle `SYS_REFCURSOR`.*
***ER Diagram Check**: ■ Matches*

68. Automated Portfolio Valuation Performance Snapshot Recorder

```
CREATE OR REPLACE FUNCTION record_portfolio_snapshot(p_portfolio_id IN RAW)
RETURN RAW
IS
    v_total_value   NUMBER(18, 2) := 0.0;
    v_total_cost    NUMBER(18, 2) := 0.0;
    v_total_pnl     NUMBER(18, 2) := 0.0;
    v_total_pnl_pct NUMBER(8, 4) := 0.0;
    v_perf_id       RAW(16);
BEGIN
    -- Calculate current portfolio valuation aggregates
    SELECT
        NVL(SUM(h.quantity * NVL(pc.price_usd, ts.price_per_unit)), 0.0),
        NVL(SUM(h.quantity * h.average_buy_price), 0.0)
    INTO v_total_value, v_total_cost
    FROM holdings h
    LEFT JOIN assets a ON h.symbol = a.symbol
    LEFT JOIN ticker_stats ts ON a.asset_id = ts.asset_id
    LEFT JOIN price_cache pc ON h.symbol = pc.symbol AND pc.expire_at > SYSTIMESTAMP
    WHERE h.portfolio_id = p_portfolio_id;

    v_total_pnl := v_total_value - v_total_cost;

    IF v_total_cost > 0 THEN
        v_total_pnl_pct := ROUND(((v_total_pnl / v_total_cost) * 100.0), 4);
    ELSE

        v_total_pnl_pct := 0.0;
    END IF;

    -- Record snapshot in portfolio_performance
    v_perf_id := SYS_GUID();
    INSERT INTO portfolio_performance (
        performance_id,
        portfolio_id,
        total_value,
        total_profit_loss,
        total_profit_loss_pct,
        timestamp
    ) VALUES (
        v_perf_id,
        p_portfolio_id,
        v_total_value,
        v_total_pnl,
        v_total_pnl_pct,
        SYSTIMESTAMP
    );

    RETURN v_perf_id;
EXCEPTION
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20020, 'Failed to record performance snapshot for portfolio: ' || SQLERRM);

END record_portfolio_snapshot;
/
```

***Purpose**: Aggregates all current asset holding values in a portfolio and persists a point-in-time equity and performance record to `portfolio_performance`.*
***ER Diagram Check**: ■ Matches*

69. Expire and Purge Stale Price Cache Routine

```
CREATE OR REPLACE FUNCTION expire_price_cache
RETURN NUMBER
IS
    v_deleted_count NUMBER := 0;
BEGIN
    DELETE FROM price_cache
    WHERE expire_at < SYSTIMESTAMP;

    v_deleted_count := SQL%ROWCOUNT;

    RETURN v_deleted_count;
EXCEPTION
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20030, 'Price cache expiration routine failed: ' || SQLERRM);
END expire_price_cache;
/
```

***Purpose**: Housekeeping cleanup routine that purges stale market quote rows from `price_cache` whose TTL timestamp has elapsed, returning the pruned row count.*
***ER Diagram Check**: ■ Matches*

70. Recalculate 24-Hour Ticker Statistics from Price History

```
CREATE OR REPLACE FUNCTION refresh_ticker_stats(p_asset_id IN RAW)
RETURN NUMBER
IS
    v_symbol       VARCHAR2(20);
    v_latest_price NUMBER(18, 6);
    v_high_24h     NUMBER(18, 6);
    v_low_24h      NUMBER(18, 6);
    v_volume_24h   NUMBER(24, 2);
BEGIN
    -- Find symbol for given asset
    BEGIN
        SELECT symbol INTO v_symbol
        FROM assets
        WHERE asset_id = p_asset_id;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20040, 'Asset with specified ID does not exist');
    END;

    -- Extract aggregates from price_history over the last 24 hours
    SELECT
        MAX(ph.price),
        MIN(ph.price),
        NVL(SUM(ph.volume), 0.0)
    INTO v_high_24h, v_low_24h, v_volume_24h

    FROM price_history ph
    WHERE (ph.asset_id = p_asset_id OR ph.symbol = v_symbol)
      AND ph.timestamp >= SYSTIMESTAMP - INTERVAL '24' HOUR;

    -- Fetch the most recent recorded price
    BEGIN
        SELECT price INTO v_latest_price
        FROM (
            SELECT price
            FROM price_history ph
            WHERE (ph.asset_id = p_asset_id OR ph.symbol = v_symbol)
            ORDER BY ph.timestamp DESC
        )
        WHERE ROWNUM = 1;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            v_latest_price := NULL;
    END;

    -- If recent history exists, upsert into ticker_stats
    IF v_latest_price IS NOT NULL THEN
        MERGE INTO ticker_stats t
        USING (
            SELECT
                p_asset_id                      AS asset_id,
```

```
                v_latest_price                  AS price_per_unit,
                NVL(v_volume_24h, 0.0)          AS volume_24h,
                NVL(v_high_24h, v_latest_price) AS high_24h,
                NVL(v_low_24h, v_latest_price)  AS low_24h,
                SYSTIMESTAMP                    AS updated_at
            FROM dual
        ) src
        ON (t.asset_id = src.asset_id)
        WHEN MATCHED THEN
            UPDATE SET
                t.price_per_unit = src.price_per_unit,
                t.volume_24h     = src.volume_24h,
                t.high_24h       = src.high_24h,
                t.low_24h        = src.low_24h,
                t.updated_at     = src.updated_at
        WHEN NOT MATCHED THEN
            INSERT (asset_id, price_per_unit, volume_24h, high_24h, low_24h, updated_at)
            VALUES (src.asset_id, src.price_per_unit, src.volume_24h, src.high_24h, src.low_24h, src.updated_at);

        RETURN 1; -- Returns 1 for TRUE
    END IF;

    RETURN 0; -- Returns 0 for FALSE
EXCEPTION
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20041, 'Failed to refresh ticker stats: ' || SQLERRM);
END refresh_ticker_stats;
/
```

***Purpose**: Computes rolling 24-hour high, low, volume, and last price statistics from historical timeseries data points in `price_history` and upserts the calculated summary into `ticker_stats`.*
***ER Diagram Check**: ■ Matches*